

A disproof of Conjecture 00000008666

The non-Hamiltonian Cayley digraphs do not form a finite list

1 The conjecture

Conjecture 00000008666 reads (English version):

Definition: Hamiltonian cycles of Cayley graphs: the Hamiltonicity of Cayley graphs of finite groups. *Conjecture:* Cayley graphs of every finite group (for every generating set) are Hamiltonian (a Hamiltonian law); the known exceptions (Cayley digraphs without Hamiltonicity) form a finite list (a finite list law); the smallest unresolved group outside the list is $\text{PSL}(2,p)$ (a PSL unresolved law); and the canonical structure of generating cycles is lexicographic type (a lexico law).

The Chinese version (in `conjectures/00000008666.md`) gives the second clause as, in transliteration, “yizhi liwai (wu Hamilton de Cayley *youxiangtu*) de liebiao youxian (youxian liebiao lv)”: “the list of known exceptions (Cayley *digraphs* without Hamilton [cycles]) is finite (finite list law)”. Both versions identify the exceptions explicitly as Cayley digraphs without a Hamiltonian cycle.

The conjecture is the conjunction of four clauses: (1) the Hamiltonian law, (2) the finite list law, (3) the PSL unresolved law, (4) the lexico law.

Result. Clause (2) is false. For every odd $n \geq 3$ the Cayley digraph of the cyclic group $\mathbb{Z}/2 \times \mathbb{Z}/n \cong \mathbb{Z}/2n$ with generators $(1, 0)$ and $(0, 1)$ has no directed Hamiltonian cycle (Rankin’s coset argument). These exceptions have unbounded order, so they do not form a finite list. Hence the conjunction $(1) \wedge (2) \wedge (3) \wedge (4)$ is false whatever clauses (1), (3) and (4) mean. Everything is formalized in Lean 4 with Mathlib (Section 5).

2 Reading

- (a) **The exceptions are Cayley digraphs.** The parenthesis in both languages is the only definition the text gives of “exceptions”, and it names a classical object: Cayley digraphs of finite groups without a Hamiltonian cycle. The standard reference records that these exist and are plentiful (Section 3). We refute the clause for exactly this object.
- (b) **“Finite list.”** If the exceptions formed a finite list $(G_1, S_1), \dots, (G_k, S_k)$ (up to isomorphism, or as a list of groups), their orders would be bounded by $\max_i |G_i|$. We refute this weakest consequence: the set of orders $|G|$ of exceptions is infinite. We even restrict to generating sets that do not contain the identity.
- (c) **“Known.”** Under the non-epistemic reading (“the exceptions form a finite list”) the Lean theorem refutes the clause directly. Under the epistemic reading (“the exceptions known today form a finite list”) the clause is also false. Rankin’s family is classical (1948/1966), the standard reference states that every cyclic group whose order is not a prime power has

a non-Hamiltonian directed Cayley graph (Section 3), and the Lean proof below establishes infinitely many exceptions as one explicit family, proved uniformly in n .

- (d) **Clause (1).** If “Cayley graphs” in clause (1) includes digraphs, clause (1) is false as well; the Lean file proves this (`directedHamiltonianLaw_false`). The undirected reading of clause (1) is the open Lovasz-type conjecture for Cayley graphs; we make no claim about it, and none is needed.

3 Definitions and sources

Definition 1 (Cayley digraph). For a group G and $S \subseteq G$, the Cayley digraph $\text{Cay}(G, S)$ has vertex set G and an arc $g \rightarrow gs$ for every $g \in G$, $s \in S$. Mathlib’s Cayley graph `SimpleGraph.mulCayley S` uses the same convention ($u \sim v$ iff $u \neq v$ and $ug = v$ or $vg = u$ for some $g \in S$); the Lean file proves that the underlying simple graph of our digraph is exactly `mulCayley S`.

Definition 2 (Directed Hamiltonian cycle). A directed Hamiltonian cycle of a finite digraph on m vertices is an enumeration $v_0, v_1, \dots, v_{m-1}$ of all vertices, each exactly once, with arcs $v_i \rightarrow v_{i+1}$ for $i < m - 1$ and $v_{m-1} \rightarrow v_0$. The digraph is Hamiltonian if it has one.

Sources (quotations verbatim):

- Wikipedia, *Lovasz conjecture*, https://en.wikipedia.org/wiki/Lov%C3%A1sz_conjecture: “Every directed Cayley graph of an abelian group has a Hamiltonian path; however, every cyclic group whose order is not a prime power has a directed Cayley graph that does not have a Hamiltonian cycle.” Also: “Various counterexamples were obtained by Robert Alexander Rankin.” Also: “None of the 5 vertex-transitive graphs with no Hamiltonian cycles is a Cayley graph.”
- R. A. Rankin, *A campanological problem in group theory*, Math. Proc. Cambridge Philos. Soc. (1948), <https://doi.org/10.1017/s030500410002394x>; part II (1966), <https://doi.org/10.1017/s0305004100039451>.
- D. Witte and J. A. Gallian, *A survey: Hamiltonian cycles in Cayley graphs*, Discrete Math. (1984), [https://doi.org/10.1016/0012-365x\(84\)90010-4](https://doi.org/10.1016/0012-365x(84)90010-4).

The argument below does not rely on these sources: it is proved in full, in Lean.

4 The proof

Lemma 3 (Rankin). *Let G be a finite abelian group and $a \neq b$ elements of G such that ab^{-1} generates G and $\text{ord}(a) < |G|$, $\text{ord}(b) < |G|$. Then $\text{Cay}(G, \{a, b\})$ has no directed Hamiltonian cycle.*

Proof. Let $m = |G|$ and suppose $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{m-1} \rightarrow v_0$ is a directed Hamiltonian cycle. Define $\sigma(v_i) = v_{i+1 \bmod m}$. This is a bijection of G , and for every x we have $\sigma(x) = xa$ or $\sigma(x) = xb$.

Key step. If $\sigma(x) = xa$, then $\sigma(xab^{-1}) = xab^{-1}a$. Otherwise $\sigma(xab^{-1}) = xab^{-1}b = xa = \sigma(x)$, so $xab^{-1} = x$ by injectivity, i.e. $a = b$, a contradiction.

By induction, $\sigma(x(ab^{-1})^k) = x(ab^{-1})^k a$ for all $k \geq 0$. Since G is finite and ab^{-1} generates it, every $y \in G$ equals $x(ab^{-1})^k$ for some $k \geq 0$, so $\sigma(y) = ya$ for all y . The same argument with a, b

exchanged (note that $ba^{-1} = (ab^{-1})^{-1}$ also generates G) shows: if one step is b , then every step is b .

So all steps equal one $c \in \{a, b\}$, and by induction $v_k = v_0 c^k$ for $k < m$. Since $0 < \text{ord}(c) < m$, we get $v_{\text{ord}(c)} = v_0$, contradicting injectivity of the enumeration. $\square$

Theorem 4 (Rankin's family). *For every odd $n \geq 3$, let $G_n = \mathbb{Z}/2 \times \mathbb{Z}/n$, $a = (1, 0)$, $b = (0, 1)$. Then $\{a, b\}$ generates G_n , $0 \notin \{a, b\}$, and $\text{Cay}(G_n, \{a, b\})$ has no directed Hamiltonian cycle.*

Proof. $|G_n| = 2n$, $\text{ord}(a) = \text{lcm}(2, 1) = 2 < 2n$ and $\text{ord}(b) = \text{lcm}(1, n) = n < 2n$; in particular $a \neq b$ and $a, b \neq 0$. The element $a - b = (1, -1)$ has order $\text{lcm}(2, n) = 2n$ because n is odd, so it generates G_n , and hence so does $\{a, b\}$. Lemma 3 applies. $\square$

Corollary 5. *The orders of finite groups G having a generating set $S \not\ni 1$ with $\text{Cay}(G, S)$ non-Hamiltonian include every $2n$ with $n \geq 3$ odd, so they form an infinite set. Hence the exceptions do not form a finite list: clause (2) is false, and so is the conjecture.*

Proof. Given B , take $n = 2B + 3$; then $2n > B$ is the order of an exception by Theorem 4. $\square$

5 Lean 4 formalization

The directory `lean/` contains a Lean 4 project (Lean v4.33.1, Mathlib v4.33.1). `Conjecture8666.lean` imports the single source file `Conjecture8666/Basic.lean` (297 lines, namespace `Conjecture8666`, only import `Mathlib`).

Definitions.

- `cayleyDigraph G S : Digraph G` with `Adj g h :=  $\exists s \in S, g * s = h$` (Mathlib's `Digraph`).
- `IsHamiltonianCycle D v` for `v : Fin (Fintype.card V) → V`: `Bijjective v  $\wedge \forall i, D.Adj (v i) (v (finRotate _ i))$` ; `IsHamiltonian D :=  $\exists v, \text{IsHamiltonianCycle } D v$` . The theorem `finRotate_val` proves `(finRotate m i).val = (i.val + 1) % m`, so this is exactly the textbook definition above.
- `Gn n := Multiplicative (ZMod 2  $\times$  ZMod n)`, `genA n := ofAdd (1, 0)`, `genB n := ofAdd (0, 1)`.
- `exceptionOrders : Set ℕ`: the set of m such that there are $G : \text{Type}$ with `[Group G] [Fintype G]` and $S : \text{Set } G$ with `Subgroup.closure S =  $\top$` , $(1 : G) \notin S$, $\neg \text{IsHamiltonian (cayleyDigraph } G \ S)$ and `Fintype.card G = m`.
- `FiniteListLaw := exceptionOrders.Finite (clause (2), reading (b))`.
- `DirectedHamiltonianLaw`: for all $G : \text{Type}$, `[Group G] [Fintype G]`, $S : \text{Set } G$, `Subgroup.closure S =  $\top$` $\rightarrow \text{IsHamiltonian (cayleyDigraph } G \ S)$.

Theorems.

- `mulCayley_adj_iff`: $(\text{SimpleGraph.mulCayley } S).\text{Adj } u \ v \leftrightarrow u \neq v \wedge ((\text{cayleyDigraph } G \ S).\text{Adj } u \ v \vee (\text{cayleyDigraph } G \ S).\text{Adj } v \ u).$
- `step_propagates`, `step_const`, `card_le_orderOf_of_const`: the three steps of the proof of Lemma 3.
- `rankin`: Lemma 3 for $[\text{CommGroup } G] [\text{Fintype } G]$, with hypotheses $a \neq b$, $\forall g, g \in \text{Subgroup.zpowers } (a * b^{-1})$, $\text{orderOf } a < \text{Fintype.card } G$, $\text{orderOf } b < \text{Fintype.card } G$, and conclusion $\neg \text{IsHamiltonian } (\text{cayleyDigraph } G \ \{a, b\})$.
- `card_Gn`, `orderOf_genA`, `orderOf_genB`, `orderOf_genA_div_genB`, `zpowers_eq_top`, `closure_gens`, `one_notMem_gens`, and `rankin_family` (`hn : Odd n`) (`h3 : 3 ≤ n`) : $\neg \text{IsHamiltonian } (\text{cayleyDigraph } (Gn \ n) \ \{\text{genA } n, \text{genB } n\})$ (Theorem 4).
- `two_mul_mem_exceptionOrders`, `exceptionOrders_infinite` : `exceptionOrders.Infinite`, `finiteListLaw_false` : $\neg \text{FiniteListLaw}$ (Corollary 5).
- `directedHamiltonianLaw_false` : $\neg \text{DirectedHamiltonianLaw}$ (reading (d)).
- The main theorem, in which `H`, `P`, `L` stand for clauses (1), (3), (4) in any reading:

`conjecture8666_false (H P L : Prop) :  $\neg (H \wedge \text{FiniteListLaw} \wedge P \wedge L)$ .`

Axiom audit. There is no `sorry`, no `native_decide` and no added axiom. `#print axioms reports`

`'Conjecture8666.conjecture8666_false'` depends on axioms:
`[propext, Classical.choice, Quot.sound]`

and the same for `exceptionOrders_infinite`, `rankin_family` and `directedHamiltonianLaw_false`. The file also elaborates with `-DwarningAsError=true`. To check:

```
cd lean
lake exe cache get
lake build
```